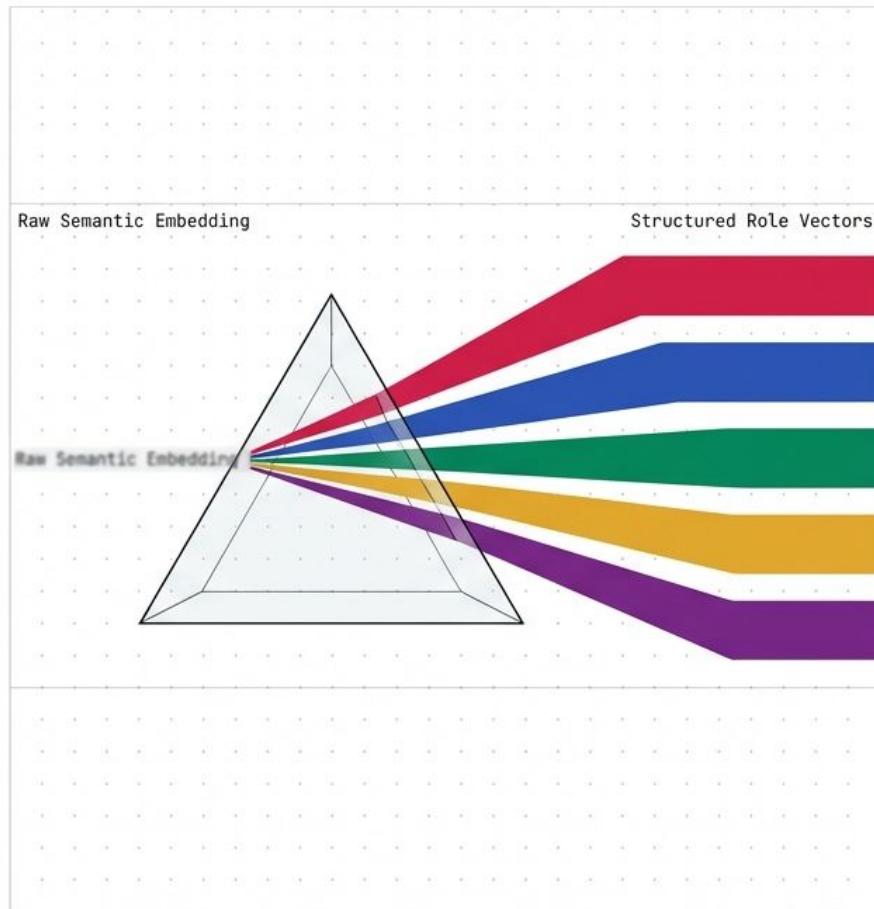


Role-Aware Log Representation

Improving Anomaly Detection
Through Latent Token Role
Induction



[illegible]

Why Manual Inspection is Impossible

Modern microservices and distributed cloud systems generate terabytes of logs daily. Systems produce millions of messages per second.

Semi-structured data. While templates are fixed, parameters like Block IDs, IPs, and locations change constantly.

Real-time detection is critical. A delay of minutes can lead to security breaches or cascading system failures.

The Evolution of Log Analysis: A Trade-off



The Semantic Leap



Era 1: Parsing-Based (DeepLog, LogRobust)

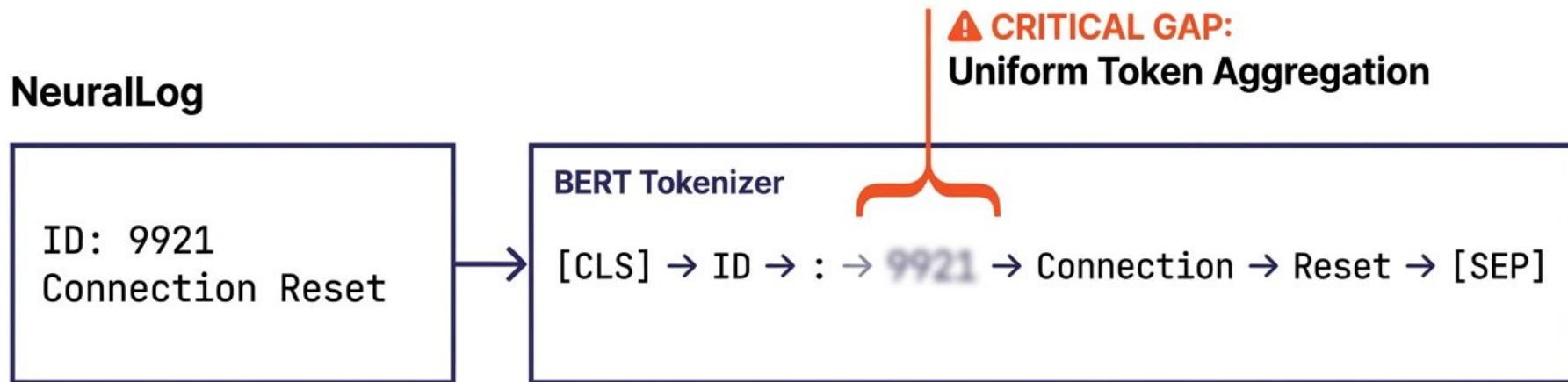
- **Method:** Extract Templates + Variables
- **Flaw:** High brittleness. Parser error rates 14-89%.
- **Consequence:** One parsing error = Detection failure.

Era 2: Parsing-Free (NeuralLog)

- **Method:** BERT encoding of raw text.
- **Win:** Robust to format changes. F1 > 0.95.
- **Flaw:** Structural Blindness (The Flattening).

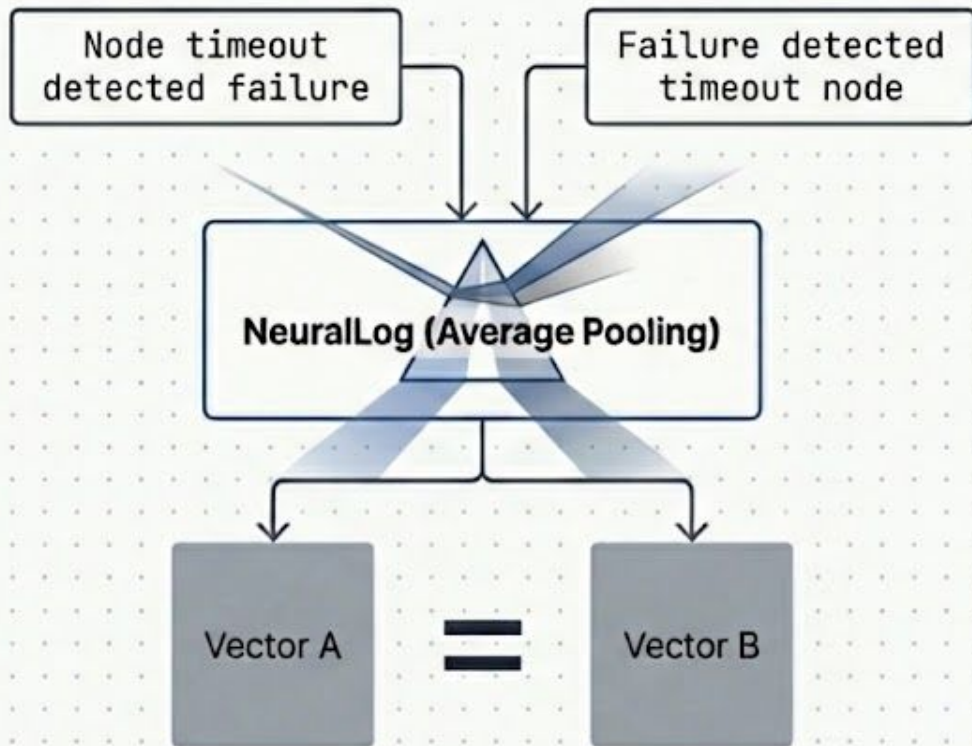
The Frontier: Parsing-Free Representation

NeuralLog



- Current State: NeuralLog encodes raw text, removing the parser.
- The Problem: Treats all tokens equally. Loss of numeric signals and structural roles.
- Need: Role-Aware frameworks that distinguish System IDs from Timestamps.

The Problem in NeuralLog Paper: The Flattening Effect



- 1. **Uniformity:** Averaging tokens disregards word order and structure.
- 2. **Number Blindness:** Removing numerics loses context (Error 500 vs 200).
- 3. **Black Box:** High accuracy, but zero interpretability of *why* an anomaly occurred.

The Evaluation Arena: 34 Million Log Messages

HDFS

Hadoop Distributed
File System

**11.17 Million
Messages**

Label Type: Block-level
Context: Distributed
Storage System

BGL

Blue Gene/L
Supercomputer

**4.74 Million
Messages**

Label Type: Line-level
Context: Numeric-heavy,
Supercomputing

Thunderbird

Sandia National Labs

**10 Million
Messages**

Label Type: Line-level
Context: High Ambiguity,
Uneven Types

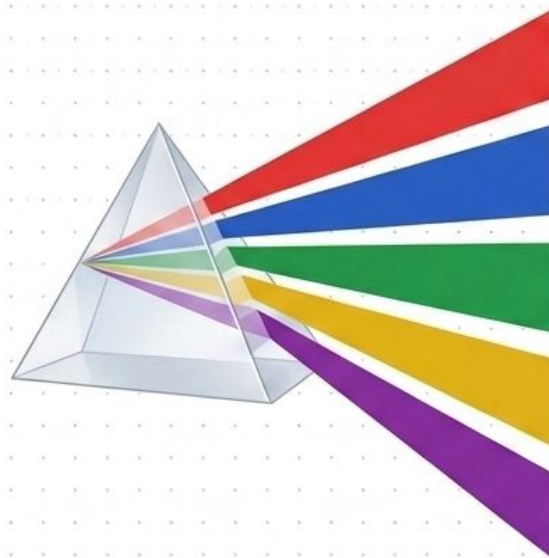
Spirit

Supercomputer Logs

**7.98 Million
Messages**

Label Type: Line-level
Context: Large Scale
Infrastructure

The Five Latent Roles

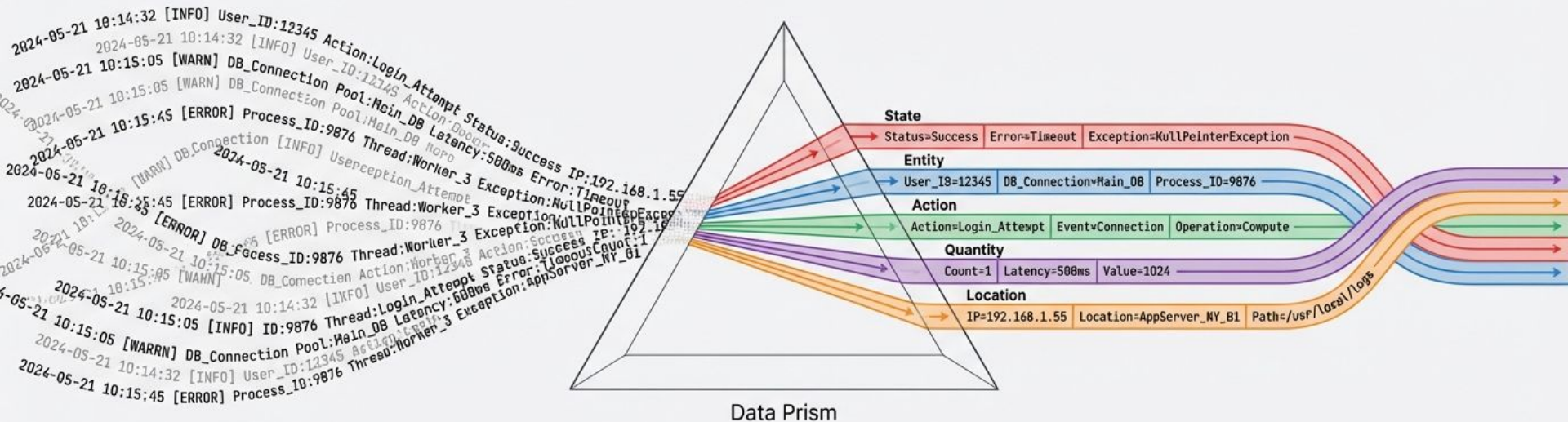


Role	Learned Tokens (BGL Dataset)
State	error, failed, timeout, interrupt
Entity	node, core, memory, processor
Action	detected, killed, terminated
Quantity	404, 500, 0x00, pid
Location	/dev, /proc, buffer

The model 'learned' these categories without human help. Operators can now see *why* an alert fired—e.g., an unexpected 'State' or abnormal 'Quantity'.

Turning Unstructured Log Noise into Semantic Structure

A Role-Aware Latent Token Induction Framework for Anomaly Detection



The Context

System health relies on logs, but raw logs are numerically dense and unstructured sequences.

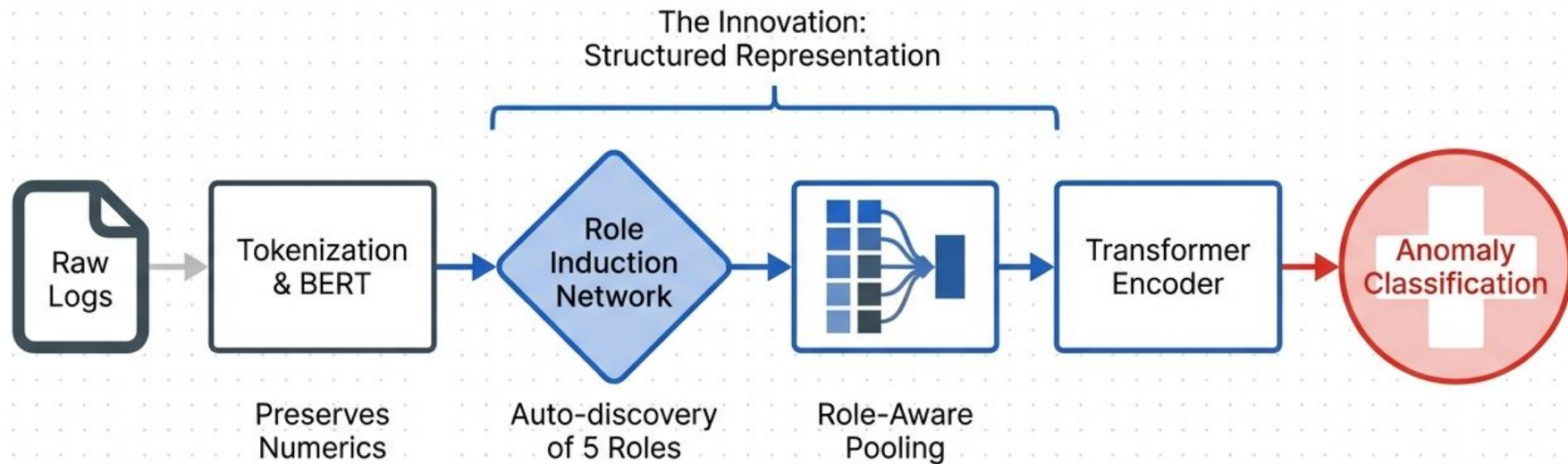
The Problem

Traditional anomaly detection averages these signals into a blur, losing critical numeric fidelity.

The Solution

We introduce a framework that identifies the functional role of every token—State, Entity, Action, Quantity, or Location—to detect anomalies with precision.

The Role-Aware Framework

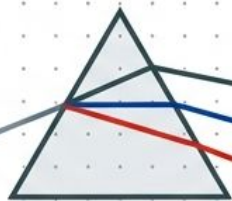


Mechanism I: Keeping the Numbers

Before & After

Standard Approach (NeuralLog)

HTTP [NUM] error
CPU usage [NUM]%
Block [NUM] failed



Our Approach

HTTP **404** error vs HTTP **500** error
CPU usage **100%**
Block **blk_1234** failed

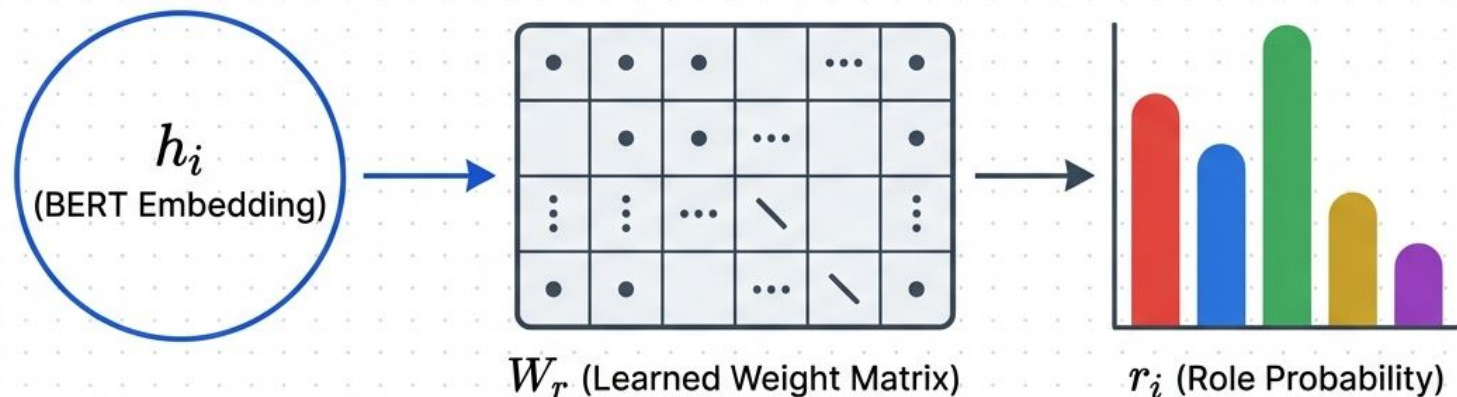
Loss of Critical Signal

Preservation of Context

Key Takeaways

- **Error Codes:** Distinguish failure modes.
- **Resource Utilization:** Distinguish idle (0%) from overload (100%).
- **Identifiers:** Enable correlation across events.

Mechanism II: Latent Token Role Induction



$$r_i = \text{softmax}(W_r \cdot \text{ReLU}(W_h h_i + b_h) + b_r)$$

- **Self-Supervised Learning:** The model learns to cluster tokens based on their contribution to the anomaly detection task.
- **No Manual Annotation:** The grammar is induced automatically from the data.

Probabilistic Mapping

Every row represents a soft probability distribution across the five structural dimensions.

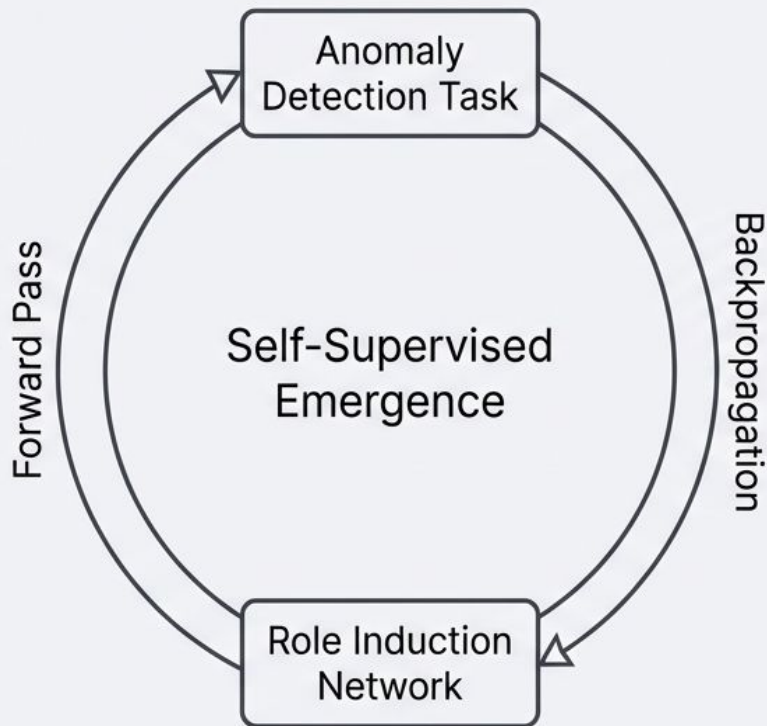
Token	State	Entity	Action	Quantity	Location	Dominant Role
CE	0.62	0.12	0.08	0.10	0.08	State
sym	0.10	0.18	0.09	0.55	0.08	Quantity
14	0.05	0.06	0.05	0.80	0.04	Quantity
location	0.08	0.10	0.07	0.05	0.70	Location
R02-M1-N1	0.05	0.12	0.06	0.08	0.69	Location
detected	0.14	0.07	0.72	0.04	0.03	Action
error	0.85	0.06	0.04	0.03	0.02	State
node	0.08	0.78	0.06	0.04	0.04	Entity
core	0.07	0.80	0.05	0.05	0.03	Entity

Emergent Classification

Notice how the highest probability in the distribution explicitly dictates the Dominant Role on the far right.

Syntax STATE ENTITY ACTION QUANTITY LOCATION

Learning Through Anomaly Detection, Not Annotation



- **Zero Manual Annotation:** We do not label tokens as 'State' or 'Action' by hand.
- **Functional Clustering:** Tokens that contribute similarly to detection (e.g., all tokens indicating failure states) naturally cluster together during training.
- **The Result:** The "roles" are learned behaviors, optimized specifically for finding system faults.

Fusing Semantics with Role Identity



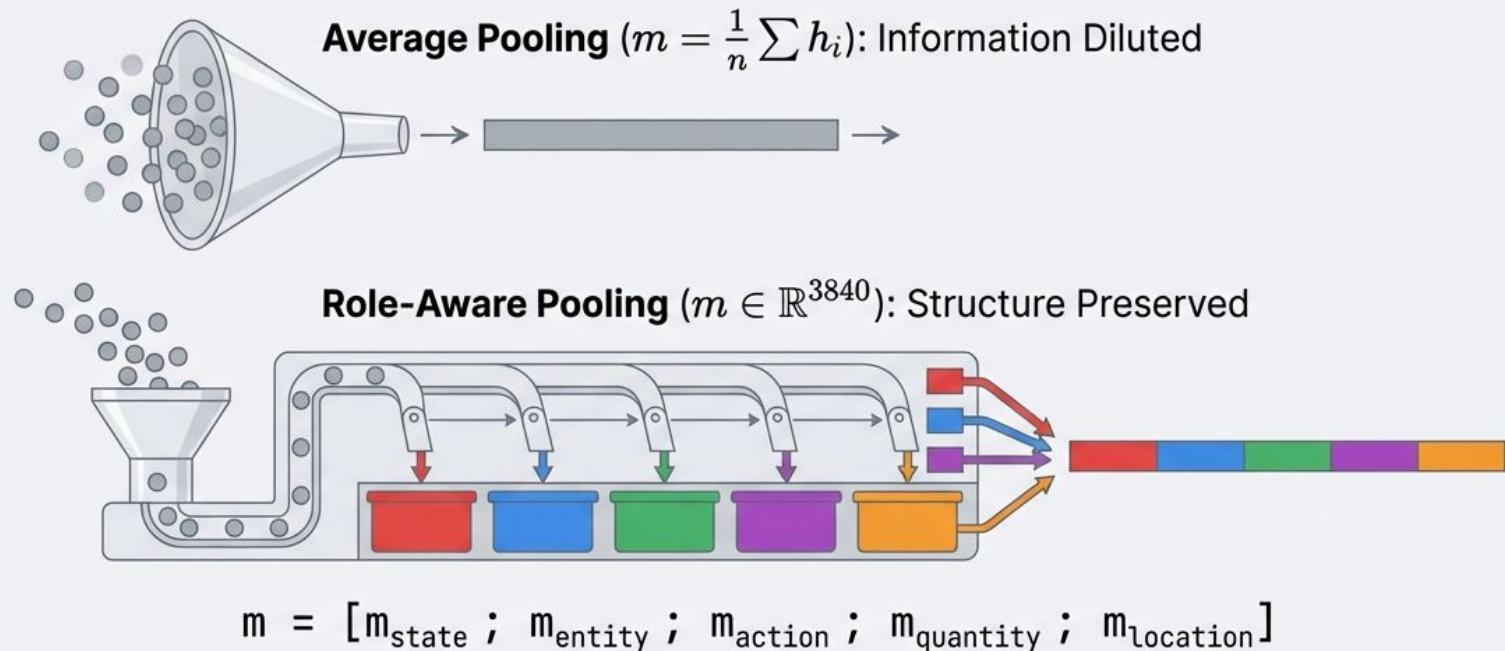
$$t_i = h_i \odot W_e r_i$$

We fuse the 'What' (Meaning) and the 'How' (Role) into a single representation.

W_e : Role Embedding Matrix ($\mathbb{R}^{768 \times 5}$)

The resulting token t_i carries both the context of the log message and the specific functional identity of the token.

Structured Pooling: Moving Beyond the Average



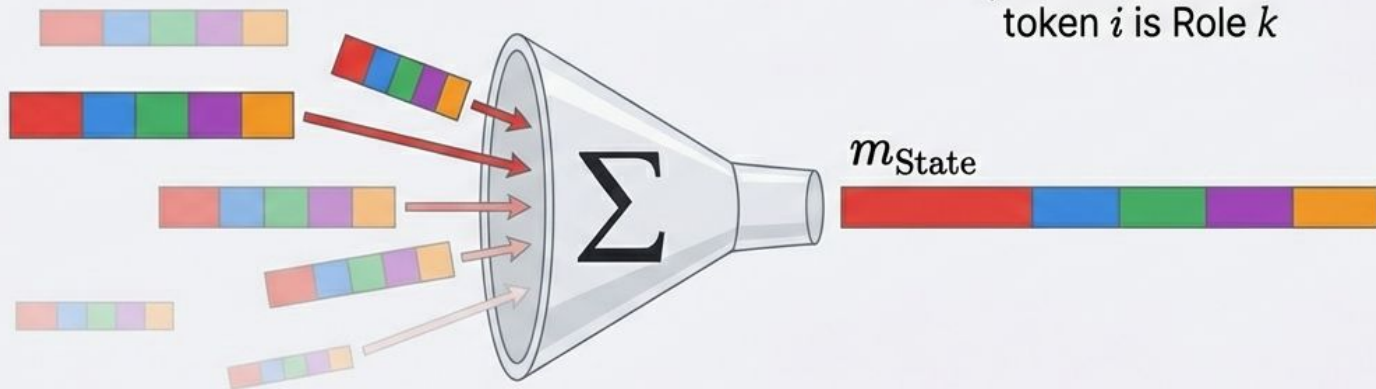
- NeuralLog simply averages embeddings, washing out signal.
- Our Method constructs a structured message vector where every dimension has a distinct purpose.
- Ensures a **Quantity** anomaly doesn't get hidden by a dominant **State** token.

The Mathematics of Role-Aware Aggregation

Segment for Role k
(e.g., **State**) $\rightarrow m_k = \frac{\sum_{i=1}^n r_{i,k} \cdot t_i}{\sum_{i=1}^n r_{i,k}}$

$\leftarrow$ Role-Aware Token

$\leftarrow$ Probability that token i is Role k



We compute a weighted average for each role. If a token has a low probability of being an **'Entity'**, it contributes almost nothing to the **Entity** segment.

Result: A precise, segmented representation of the log message.

Mechanism III: Role-Aware Pooling

Average Pooling



768 Dimensions (Blended Signal)



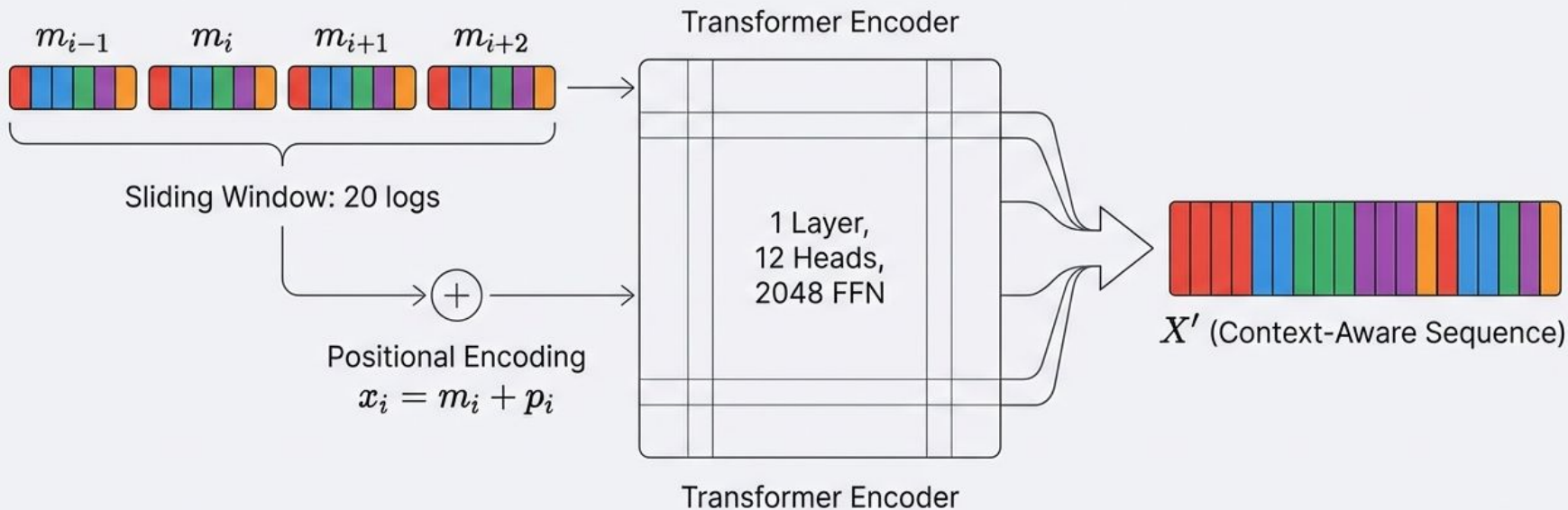
Role-Aware Pooling



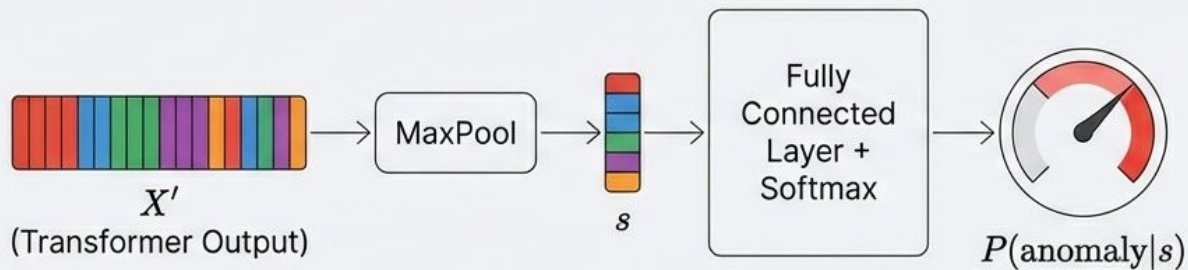
3840 Dimensions (5 Roles x 768). Preserves Hierarchical Structure.

$$m = [m_{\text{state}}; m_{\text{entity}}; m_{\text{action}}; m_{\text{quantity}}; m_{\text{location}}]$$

Modeling Temporal Dependencies



From Sequence to Anomaly Detection



Training Recipe

Loss Function: Cross-Entropy

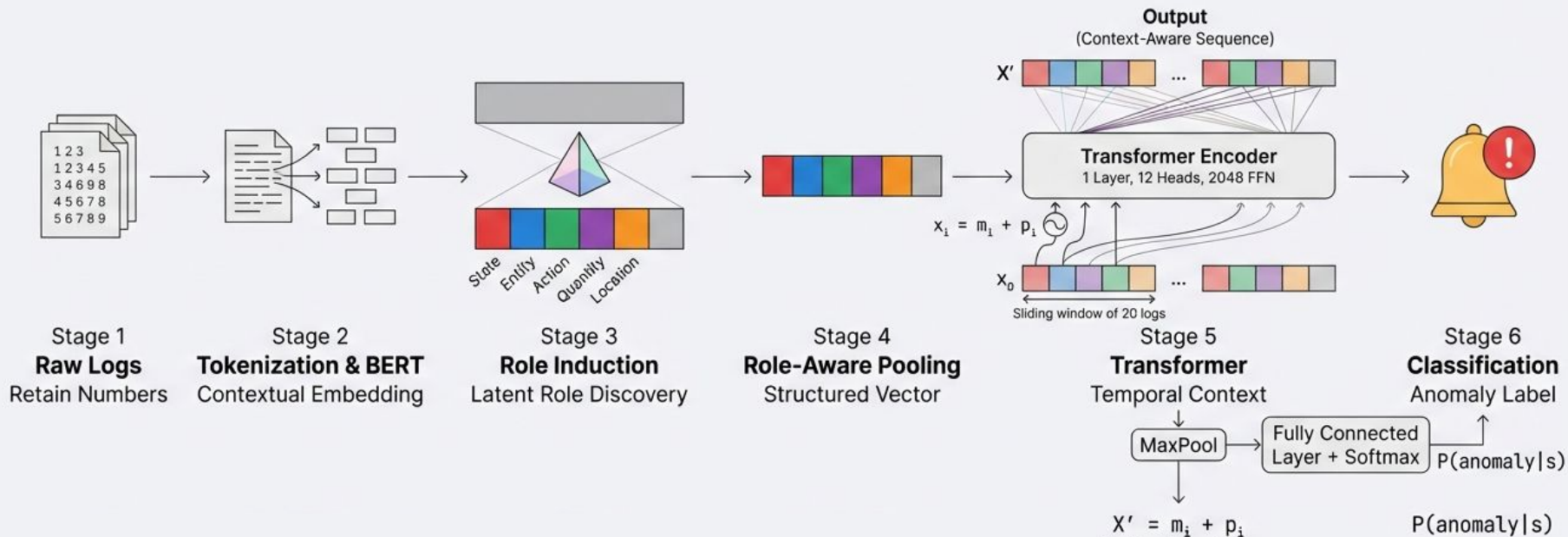
Optimizer: AdamW ($lr = 3 \times 10^{-4}$)

Batch Size: 64

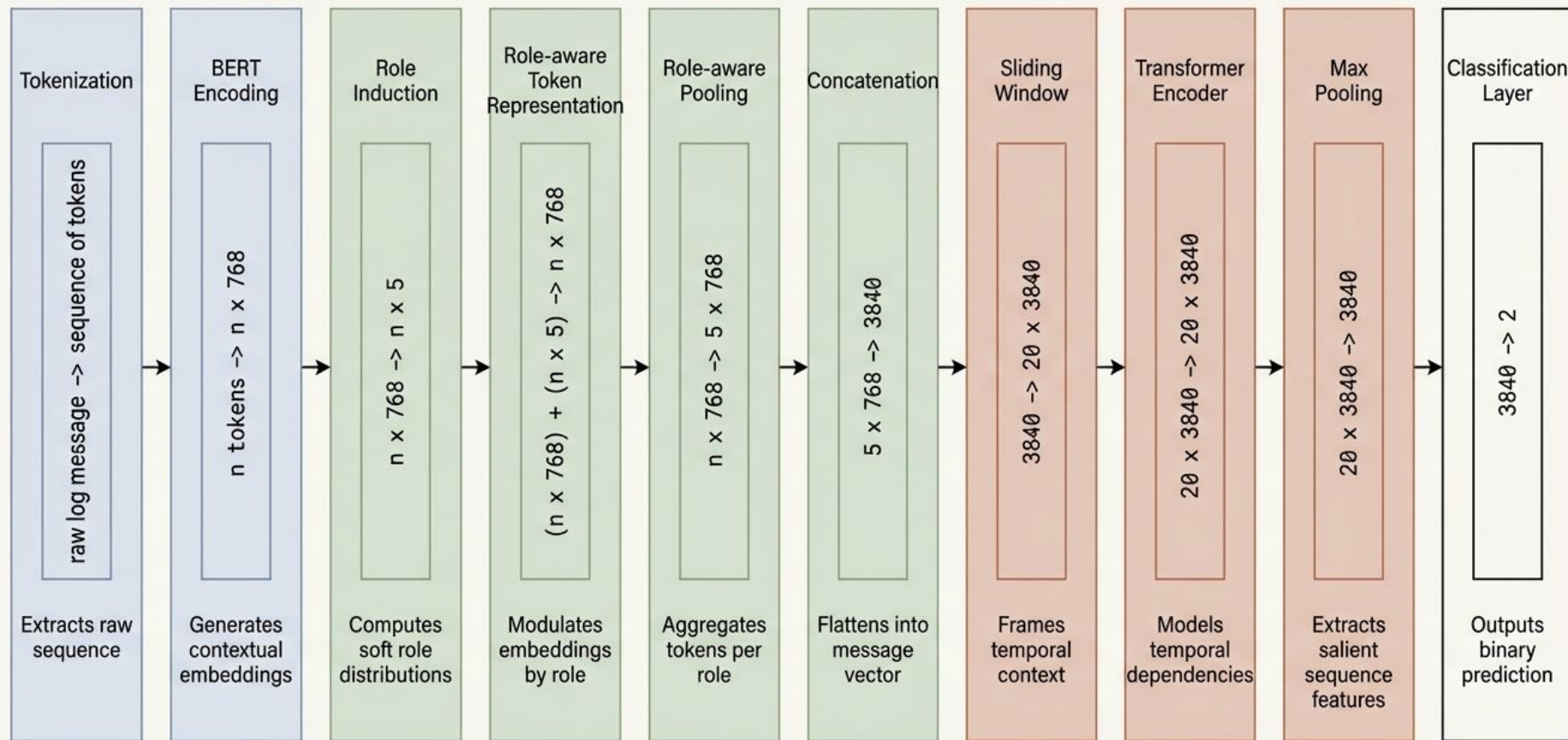
Dropout: 0.1

Early Stopping: 5 epochs (Max 20)

The Complete Architecture: From Raw Log to Decision



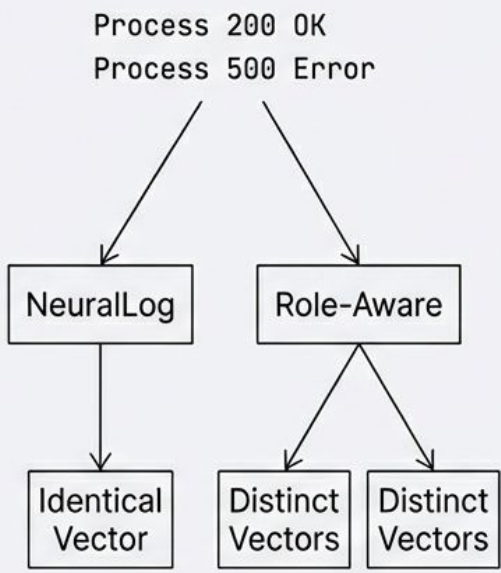
Dimensional Flow Through Pipeline



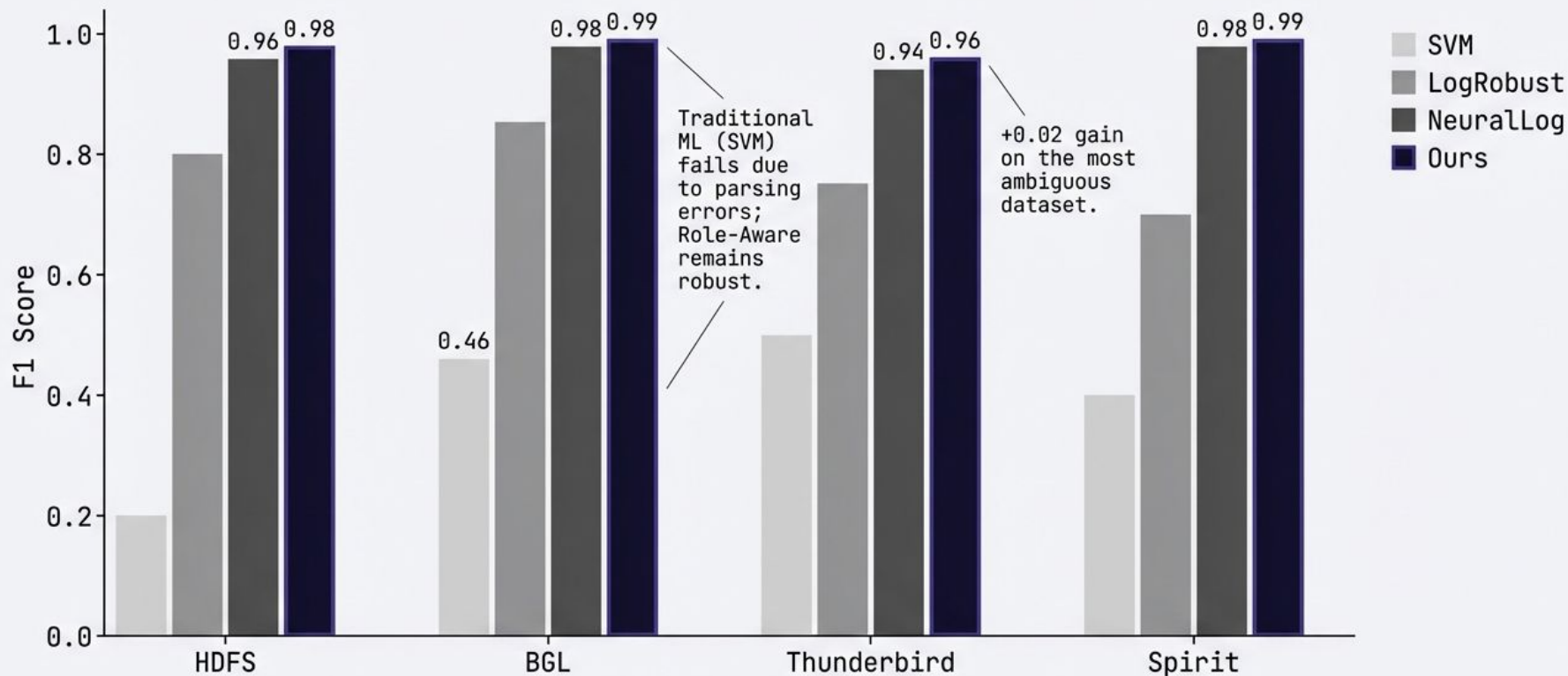
The Current Standard Has a Structural Blind Spot

Comparing NeuralLog (Baseline) vs. Role-Aware Representation

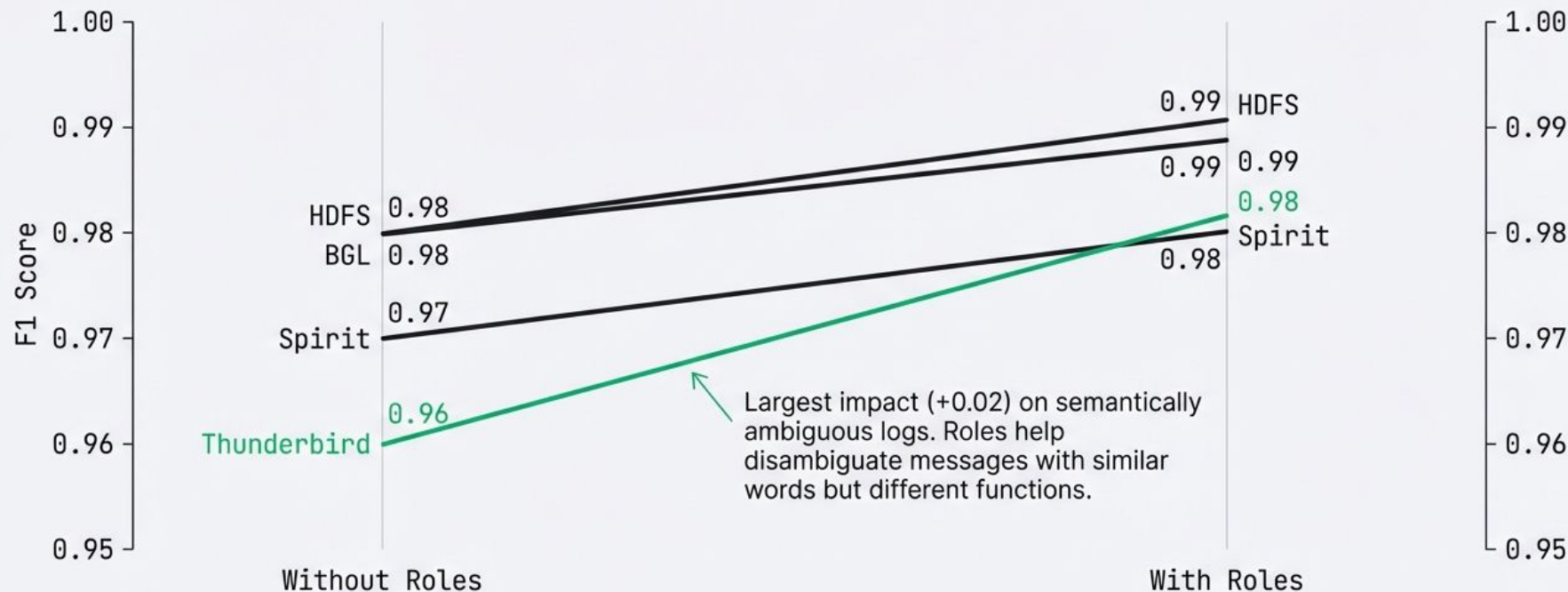
Feature	NeuralLog (Baseline)	Role-Aware (Ours)
Token Treatment	Removes Numbers (Blind to Error Codes)	Preserves Numbers (Context Aware)
Message Pooling	Uniform Average (Smeared Meaning)	Role-Separated (distinct semantic channels)
Structure Modeling	Implicit / Flat	Explicit / Structured
Vector Dimension	Limited (768 Dimensions)	Enhanced (3840 Dimensions)



Dominating Benchmarks with SOTA F1 Scores



Ablation Study: Do Roles Actually Help?



Conclusion: Removing role modeling consistently degrades performance across all datasets.

The Thunderbird Breakthrough

$$\Delta_{\text{err}}^{\text{Thunderbird}} = \frac{1 - 0.98}{1 - 0.96} = 0.50$$

Role-Aware F1-Score

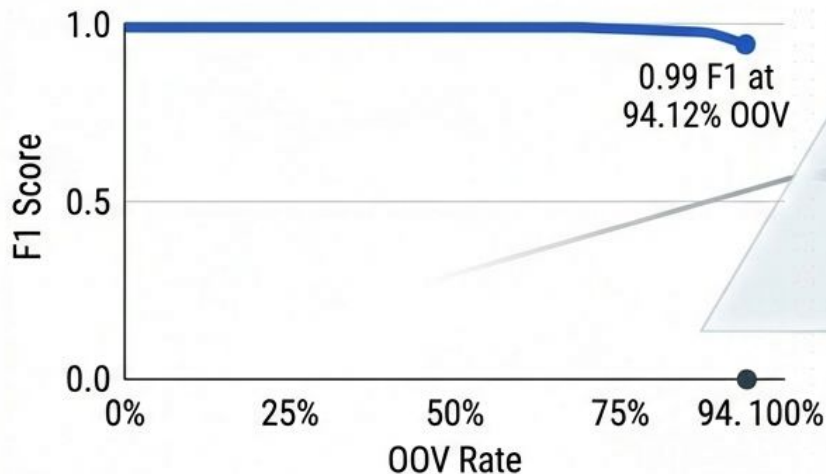
Previous SOTA F1-Score (NeuralLog)

50% relative error reduction

Halving the total volume of undetected anomalies and false alarms on a 10-million message dataset.

Robustness & Efficiency

The OOV Challenge



Maintains 0.99 F1 score even with 94.12% Out-of-Vocabulary rate on BGL.

Computational Cost

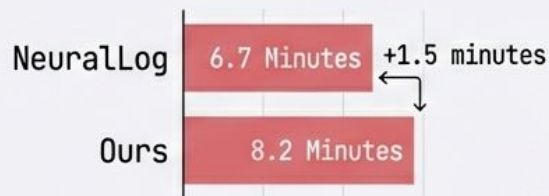
Task	NeuralLog	Ours	Delta
Preprocessing	14.3m	15.8m	+1.5m
Inference	3.1ms	3.8ms	+0.7ms

Negligible overhead compared to parsing-based methods, with significantly higher stability.

Limitations & Future Scope

Complexity

The Role Induction Network adds parameters, making the model slightly heavier than NeuralLog to train (+1.5 minutes).



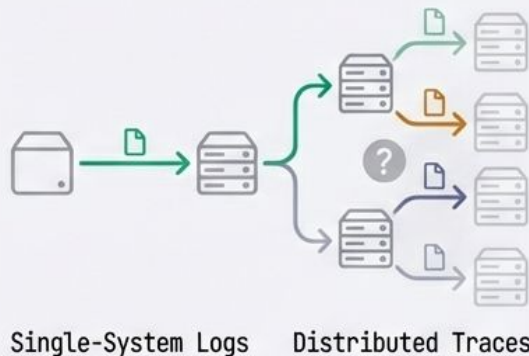
Static Roles

We utilize a fixed set of 5 roles. While empirically effective for these datasets, the optimal number might vary by domain.



System Scope

Evaluation performed on single-system logs. Distributed traces with cross-system dependencies have not yet been evaluated.



Role-Aware Log Representations for Enhanced Anomaly Detection

CONTRIBUTION OVERVIEW

- Introduces an unsupervised framework modeling token-level functional roles in raw logs
- Replaces uniform token processing with structure-preserving representation mechanisms
- Achieves state-of-the-art F1-scores (0.98–0.99) across four benchmark datasets

METHOD SUMMARY

- **Role extraction from logs:** Induces functional categories via an unsupervised neural network
- **Representation construction:** Aggregates distinct embeddings using role-aware pooling
- **Anomaly detection improvement:** Applies transformer encoders to capture temporal dependencies

KEY ADVANTAGES

- Bypasses rigid template parsing frameworks and their subsequent error propagation
- Preserves critical numeric telemetry previously discarded by traditional methods
- Provides explicit predictive interpretability through functional role attribution